CP Generator: Formal Notes

Allan Pan

May 1, 2026

1 The implemented mathematical object

The project works with a square sheet

$$S = [0, s]^2, \qquad s > 0,$$

and with a finite straight-line graph drawn on that sheet. More precisely, the implemented state is a triple

$$\mathcal{C} = (G, X, \tau),$$

where $G = (V, E)$ is a finite graph, $X \colon V \to S$ is an embedding, and

$$\tau \colon E \to \{-1, 0, 1\}$$

is a fold-label map, with -1 for "unassigned", 0 for "mountain", and 1 for "valley".

Definition 1.1 (Boundary and interior vertices)**.**

$$V_\partial = \{v \in V : X_v \in \partial S\}, \qquad V_{\text{int}} = V \setminus V_\partial.$$

Every local theorem used by the program is expressed at interior vertices, so the distinction between V_∂ and V_{int} is structural rather than cosmetic.

Definition 1.2 (Local angle data)**.** *For $v \in V_{\text{int}}$, let*

$$d(v) = \deg_G(v).$$

If the incident creases at v are listed in cyclic order, then the corresponding sector angles are written

$$\alpha_1(v), \ldots, \alpha_{2m(v)}(v) \in (0, 2\pi), \qquad \sum_{i=1}^{2m(v)} \alpha_i(v) = 2\pi.$$

Definition 1.3 (Kawasaki residual and Maekawa balance)**.** *For $v \in V_{\text{int}}$, define*

$$\kappa(v) = \max\left(\left|\textstyle\sum_{\text{i odd}} \alpha_i(v) - \pi\right|, \left|\textstyle\sum_{\text{i even}} \alpha_i(v) - \pi\right|\right).$$

If τ is complete at v, define

$$M_\tau(v) = \#\{e \ni v : \tau(e) = 0\}, \qquad V_\tau(v) = \#\{e \ni v : \tau(e) = 1\}, \qquad \mu_\tau(v) = M_\tau(v) - V_\tau(v).$$

2 From random points to a planar candidate graph

The first stage of the project is purely geometric. One samples interior points

$$P \subset (0, s)^2,$$

adjoins the four corners of the square, and then projects points lying within a fixed threshold $\delta > 0$ of the boundary onto ∂S. Thus one obtains

$$P^{\square} = P \cup \{(0,0), (s,0), (s,s), (0,s)\}, \qquad \widetilde{P} = \Pi_{\partial,\delta}(P^{\square}).$$

The candidate graph is then taken from the Delaunay triangulation of $\widetilde{P}$:

$$G_0 = \mathrm{Del}(\widetilde{P}).$$

After this, the code removes every edge lying entirely on one side of the square boundary. Therefore the implemented crease graph is

$$E = E(G_0) \setminus E_{\partial},$$

where

$$E_{\partial} = \{\{u, v\} \in E(G_0) : X_u, X_v \text{ lie on one boundary side of } S\}.$$

This Delaunay stage is not itself a flat-foldability theorem. Its role is to provide a planar graph with reasonably regular local geometry so that the later angle constraints are numerically meaningful.

3 The local flat-foldability conditions

The next stage turns the geometric candidate graph into a constrained origami object by imposing the standard single-vertex necessary conditions.

Theorem 3.1 (Single-vertex necessary conditions [2])**.** *If $v \in V_{\mathrm{int}}$ is flat-foldable as a single-vertex crease pattern, then*

$$d(v) \in 2\mathbb{Z}, \qquad \sum\nolimits_{\mathrm{i\ odd}} \alpha_i(v) = \pi, \qquad \sum\nolimits_{\mathrm{i\ even}} \alpha_i(v) = \pi, \qquad |\mu_\tau(v)| = 2.$$

The optimizer in the code imposes only one alternating-angle equality per vertex, not two. This is sufficient, because the second equality is redundant.

Lemma 3.2 (Alternating-sum redundancy)**.** *If*

$$\sum_{i=1}^{2m} \alpha_i = 2\pi,$$

then

$$\sum\nolimits_{\mathrm{i\ odd}} \alpha_i = \pi \iff \sum\nolimits_{\mathrm{i\ even}} \alpha_i = \pi.$$

Proof. Since

$$\sum\nolimits_{\mathrm{i\ even}} \alpha_i = 2\pi - \sum\nolimits_{\mathrm{i\ odd}} \alpha_i,$$

each equality implies the other. □

Accordingly, the project defines its local diagnostic predicate by

$$\mathrm{Local}(\mathcal{C}; \varepsilon) \iff \forall v \in V_{\mathrm{int}} \ \big(d(v) \in 2\mathbb{Z} \ \wedge \ \kappa(v) \le \varepsilon\big).$$

This is the mathematical content of the program's *local status*.

4 Parity repair before optimization

The first obstruction to local flat-foldability is odd degree. Let

$$O(G) = \{v \in V : d(v) \notin 2\mathbb{Z}\}$$

denote the odd-degree set. The implementation repairs parity by repeatedly deleting shortest paths between odd vertices.

Definition 4.1 (Implemented parity-repair step)**.** *Choose*

$$u \in O(G), \qquad w = \operatorname*{arg\,min}_{z \in O(G)\setminus\{u\}} d_G(u, z),$$

and let

$$\gamma = (u = v_0, v_1, \ldots, v_k = w)$$

be a shortest path from u to w in G. Replace E by

$$E \setminus \{\{v_{i-1}, v_i\} : 1 \le i \le k\}.$$

The reason this step is mathematically natural is that path deletion affects parity only at the endpoints.

Proposition 4.2 (Parity effect of one deletion)**.** *If the path γ above is deleted, then*

$$\deg(v_i) \mapsto \deg(v_i) - 2 \qquad (1 \le i \le k-1),$$

and

$$\deg(u) \mapsto \deg(u) - 1, \qquad \deg(w) \mapsto \deg(w) - 1.$$

Hence the parity of every internal path vertex is preserved, while the parity of the endpoints flips.

Proof. Each internal path vertex loses exactly two incident edges, and each endpoint loses exactly one. □

This proves that the repair has the intended local parity effect. It does not prove that the repaired graph is globally optimal in any stronger sense.

5 Weighted geometric repair via constrained optimization

Once parity has been repaired, the code moves the vertices as little as possible while imposing Kawasaki's identity at every interior vertex.

Let $X^{(0)} = (X_v^{(0)})_{v \in V}$ denote the pre-optimization coordinates. Boundary motion is heavily penalized by weights

$$w_v = \begin{cases} 10^6, & v \in V_\partial, \\ 1, & v \in V_{\text{int}}. \end{cases}$$

The weighted displacement functional is

$$D(X) = \sum_{v \in V} w_v \left\| X_v - X_v^{(0)} \right\|^2.$$

The actual code minimizes

$$F(X) = \frac{0.1}{2} D(X)^2.$$

Since $D(X) \geq 0$, one has

$$\arg\min F = \arg\min D.$$

Thus the implementation still solves the nearest-feasible-geometry problem in the weighted least-squares sense.

For each interior vertex v, define the constraint

$$g_v(X) = \sum\nolimits_{\text{i odd}} \alpha_i(v; X) - \pi.$$

The implemented optimization problem is therefore

$$\min_X F(X) \qquad \text{subject to} \qquad g_v(X) = 0 \quad \forall v \in V_{\text{int}}. \tag{1}$$

The solver is SLSQP. After the numerical solve, vertices within a small tolerance of the boundary are snapped back onto ∂S. In this way, the project passes from a merely planar sample to a planar sample constrained by the local angle theorem.

6 Mountain–valley search by Bern–Hayes-style local reduction

After geometric repair, the remaining local problem is to assign mountain and valley labels. The code does not search over all edge labelings directly. Instead, it first extracts local sign relations from smallest sectors, following the Bern–Hayes reduction idea [1].

For an interior vertex v, let

$$e_1, \ldots, e_r$$

be the incident creases in cyclic order, and let

$$\beta_1, \ldots, \beta_r$$

be the corresponding sector angles. The reduction repeatedly removes a locally minimal sector and records whether its two boundary creases must carry the same or opposite mountain–valley sign.

Definition 6.1 (Local reduction chain)**.** *The reduction at v produces*

$$\Gamma_v = \big((f_1, \ldots, f_k), (\eta_1, \ldots, \eta_k)\big), \qquad \eta_i \in \{0, 1, 2\},$$

where

$$\eta_i = \begin{cases} 0, & \tau(f_i) = 1 - \tau(f_{i-1}), \\ 1, & \tau(f_i) = \tau(f_{i-1}), \\ 2, & \tau(f_i) \text{ remains undetermined.} \end{cases}$$

Boundary vertices are treated similarly, except that temporary virtual boundary rays are added so that the cyclic order closes. Thus the local reduction stage produces one or more chains of sign relations at every vertex.

7 Exact enumeration after chain merging

The local chains are then merged across shared creases. If the merged chains are

$$\Lambda_1, \ldots, \Lambda_g,$$

then each Λ_j contributes one free binary choice, namely the sign of its first crease. Therefore the remaining search space is

$$\{0,1\}^g .$$

For

$$c = (c_1, \ldots, c_g) \in \{0,1\}^g ,$$

the code propagates the implied signs along each chain:

$$\tau_c(e_{j,1}) = c_j,$$

and then

$$\eta_{j,i} = 0 \Longrightarrow \tau_c(e_{j,i}) = 1 - \tau_c(e_{j,i-1}),$$
$$\eta_{j,i} = 1 \Longrightarrow \tau_c(e_{j,i}) = \tau_c(e_{j,i-1}),$$
$$\eta_{j,i} = 2 \Longrightarrow \tau_c(e_{j,i}) = -1.$$

This yields a partial or complete labeling τ_c. The exact admissibility filter is Maekawa's theorem:

$$\mathrm{Adm}(c) \Longleftrightarrow \forall v \in V_{\mathrm{int}} \ \ |\mu_{\tau_c}(v)| = 2.$$

Among admissible choices, the code chooses the one maximizing an interlacing score

$$\mathrm{Score}(c) = 0.05 \, \# \{e \in E : \tau_c(e) \in \{0,1\}\} \sum_{v \in V} \Phi_v(c),$$

where $\Phi_v(c)$ rewards alternating mountain–valley patterns more than repeating ones, with sharper wedges weighted more heavily. This score is not a correctness condition; it is only a tie-breaker among already admissible assignments.

Proposition 7.1 (What the assignment search certifies)**.** *If the routine returns an assignment τ^*, then*

$$\forall v \in V_{\mathrm{int}} \ \ |\mu_{\tau^*}(v)| = 2.$$

If the routine returns failure, then no enumerated merged-group assignment satisfies Maekawa's theorem at every interior vertex.

Proof. The code checks every seed choice in $\{0,1\}^g$, rejects every choice violating Maekawa, and returns only from the surviving set. □

8 From local certificates to global geometry

At this point, the project has local angle control and a locally admissible mountain–valley labeling. The next question is global: whether the current straight-line embedding can be organized into coherent folded faces.

The first global obstruction is crossing.

Definition 8.1 (Crossing predicate)**.** *For distinct nonincident edges $e, e' \in E$, define*

$$\mathrm{Cross}(e, e'; X) \iff X(e) \cap X(e') \neq \varnothing.$$

Set

$$\mathrm{Noncross}(X) \iff \forall e \neq e' \in E, \quad e \cap e' = \varnothing \implies \neg\, \mathrm{Cross}(e, e'; X).$$

If $\mathrm{Noncross}(X)$ fails, then no exact face-based folded preview is attempted on the current geometry.

9 Exact face extraction by half-edge tracing

Assume now that the current embedding is noncrossing. The code augments the fold graph by the four boundary edges of the square:

$$E^{\square} = E \cup E_{\partial S}.$$

At each vertex v, neighbors are ordered by polar angle around X_v. This induces a left-face successor rule on oriented edges.

Definition 9.1 (Left-face successor)**.** *For an oriented edge (u, v), let*

$$\sigma(u, v) = (v, w),$$

where w is the predecessor of u in the cyclic order around v.

Starting from any half-edge and iterating σ yields a closed walk or a failure. Every bounded counterclockwise closed walk is declared to be a face. Repeated vertices or nonclosing walks cause the exact solver to abort.

Thus the code produces a finite face family

$$\mathcal{F}.$$

For each fold edge $e \in E$, one then computes its face-incidence set

$$\mathrm{Inc}(e) = \{f \in \mathcal{F} : e \subset \partial f\}.$$

The exact solver requires

$$|\mathrm{Inc}(e)| = 2 \qquad \forall e \in E.$$

This is the precise global topological condition needed before reflection propagation can begin.

10 Reflection propagation and exact-current-geometry certification

Suppose every fold edge borders exactly two faces. The code chooses a root face

$$f_0 \in \mathcal{F}, \qquad \mathrm{area}(f_0) = \max_{f \in \mathcal{F}} \mathrm{area}(f),$$

and sets

$$T_{f_0} = I_3.$$

If faces f and g share a fold line ℓ, then the child transform is defined by reflection:

$$T_g = R_\ell T_f.$$

When a face is reached by two different reflection paths, the resulting transforms are compared on the vertices of that face.

Definition 10.1 (Cycle discrepancy)**.** *For a face f and two candidate transforms T, T', define*

$$\Delta(f; T, T') = \max_{v \in \mathrm{Vert}(f)} \left\| T(X_v) - T'(X_v) \right\|.$$

If

$$\Delta(f; T, T') > \Delta_{\mathrm{hard}},$$

then exact reconstruction fails. If

$$\Delta_{\mathrm{soft}} < \Delta(f; T, T') \leq \Delta_{\mathrm{hard}},$$

then the reconstruction survives only at warning level.

Fold signs are resolved from τ by

$$\tau(e) = 0 \Longrightarrow \mathrm{sgn}(e) = +1, \qquad \tau(e) = 1 \Longrightarrow \mathrm{sgn}(e) = -1.$$

If a fold remains unassigned, the exact solver fills its sign provisionally by a face-depth rule; this also downgrades the output to warning level.

Theorem 10.2 (Exact-current-geometry certificate)**.** *If the project reports*

$$\mathrm{Status}_{\mathrm{global}}(\mathcal{C}) = \mathrm{Pass} \qquad \textit{and} \qquad \mathrm{Status}_{\mathrm{preview}}(\mathcal{C}) = \mathrm{Pass},$$

then

$$\mathrm{Noncross}(X), \qquad |\mathrm{Inc}(e)| = 2 \ \forall e \in E, \qquad \Delta(f; T, T') \leq \Delta_{\mathrm{soft}}$$

for every cycle comparison, and no provisional fold signs were used.

Proof. Each clause is the negation of one of the exact solver's failure or warning branches. A pass can occur only if every one of those branches is avoided. □

This theorem describes the strongest global statement currently certified by the code. It is a theorem about the *current embedding*, not a complete classification theorem for all embeddings with the same combinatorics.

11 3D folding and fallback preview

Once exact face reconstruction is available, the project builds a kinematic 3D folding by propagating rotations along the face-adjacency tree. If a crease e has axis (p, q) and sign $\mathrm{sgn}(e) \in \{\pm 1\}$, then the 3-dimensional transform satisfies

$$T_g^{(3D)} = R_{(p,q)}\big(\mathrm{sgn}(e)\theta\big) T_f^{(3D)}.$$

This rigid-hinge viewpoint is the standard rigid-plate abstraction used for the preview stage; compare the rigid-origami literature [3].

If exact face reconstruction fails, the project falls back to a triangle-mesh preview. In that mode the domain is triangulated and propagated along a tree of triangle adjacencies. This is still useful visually, but it is intentionally marked as non-certifying.

12 Status predicates

The four displayed statuses are designed to keep the local and global statements separate.

Definition 12.1 (Implemented status semantics)**.**

$$\mathrm{Status}_{\mathrm{local}}(\mathcal{C}) = \begin{cases} \mathrm{Pass}, & \mathrm{Local}(\mathcal{C};\varepsilon), \\ \mathrm{Fail}, & \textit{otherwise}, \end{cases}$$

$$\mathrm{Status}_{\mathrm{assign}}(\mathcal{C}) = \begin{cases} \mathrm{Fail}, & \exists v \in V_{\mathrm{int}} : |\mu_\tau(v)| \neq 2 \textit{ with complete local assignment data}, \\ \mathrm{NotRun}, & \forall e \in E : \tau(e) = -1, \\ \mathrm{Warn}, & \exists e \in E : \tau(e) = -1 \textit{ and some assignments exist}, \\ \mathrm{Pass}, & \mathrm{Assign}(\mathcal{C}), \end{cases}$$

$$\mathrm{Status}_{\mathrm{global}}(\mathcal{C}) = \begin{cases} \mathrm{Fail}, & \neg\,\mathrm{Noncross}(X), \\ \mathrm{NotRun}, & E = \varnothing, \\ \mathrm{Unknown}, & \mathrm{Status}_{\mathrm{assign}}(\mathcal{C}) = \mathrm{NotRun}, \\ \mathrm{Pass}, \mathrm{Warn}, \mathrm{Fail}, & \textit{exact-current-geometry solver output}, \end{cases}$$

$$\mathrm{Status}_{\mathrm{preview}}(\mathcal{C}) = \begin{cases} \mathrm{NotRun}, & \textit{no preview attempt admissible}, \\ \mathrm{Pass}, \mathrm{Warn}, \mathrm{Fail}, & \textit{preview solver output}. \end{cases}$$

13 Scope and limitations

The project combines rigorous local constraints with a partly rigorous and partly heuristic global pipeline. The following distinctions are therefore essential.

- The shortest-path parity repair is an implemented heuristic, not an optimality theorem.
- The Bern–Hayes-style wedge reduction is a local sign-reduction device derived from [1], not a complete multi-vertex flat-foldability classifier.
- The exact face-reflection solver certifies the current embedding when it passes, but the mesh fallback is only a guarded visualization.
- Warning-level outputs indicate exactly where the code leaves the realm of strict certification.

Theorem 13.1 (Implemented scope)**.** *The current pipeline is not a complete decision procedure for arbitrary multi-vertex global flat-foldability.*

Proof. The claim follows from the coexistence of heuristic parity repair, local-only sign reduction, and explicit mesh/reference fallbacks in the preview stage. □

References

[1] Marshall Bern and Barry Hayes. *The Complexity of Flat Origami.* SODA, 1996.

[2] Thomas C. Hull. *On the Mathematics of Flat Origamis.* Congressus Numerantium, 1994.

[3] Zachary Abel, Jason Cantarella, Erik D. Demaine, David Eppstein, Thomas C. Hull, Jason S. Ku, Robert J. Lang, and Tomohiro Tachi. *Rigid Origami Vertices: Conditions and Forcing Sets*. Journal of Computational Geometry, 2017.

[4] David Eppstein. *A Parameterized Algorithm for Flat Folding*. arXiv:2306.11939, 2023.

[5] Thomas C. Hull and Inna Zakharevich. *Flat Origami Is Turing Complete*. arXiv:2309.07932, version consulted in 2025.

[6] Andreas Walker and Tino Stanković. *Algorithmic Design of Origami Mechanisms and Tessellations*. Communications Materials, 2022.